How Redundant Are Transformer Layers?
Comparing Attention and MLP Layers Redundancy in Large Language Models

Cochiorca Oana Maria **Lungu Laura-Vanesa** **Negoiță-Crețu Raluca-Marina**

oana-maria.cochiorca@s.unibuc.ro laura-vanesa.lungu@s.unibuc.ro raluca-marina.negoita-cretu@s.unibuc.ro

University of Bucharest, Faculty of Mathematics and Computer Science
Bucharest, Romania

Abstract

As LLMs move into real-world applications, their high computational cost poses a major deployment challenge. Recent work suggests that Transformer architectures contain significant redundancy, particularly in attention layers. We investigate whether this redundancy extends beyond LLMs to fine-tuned multilingual and encoder–decoder architectures. We conduct systematic layer ablations on RoLLaMA2-7B, TinyLlama, and mT5-small, progressively removing attention and MLP layers while evaluating performance on reasoning benchmarks. Our results show that attention redundancy generalizes across architectures: RoLLaMA2-7B preserves approximately 91% of its performance after removing 37.5% of attention layers. These findings suggest that attention redundancy is a fundamental property of Transformers and can be exploited to improve efficiency with minimal performance loss.

1 Introduction

In this work, we build on the findings of He et al. (2024), who show that attention layers in large decoder-only English models exhibit substantial redundancy, allowing many of them to be removed with limited performance loss. However, it remains unclear whether this behavior extends beyond such settings. We therefore study whether attention-layer redundancy generalizes across **model scale**, **language adaptation**, and **architecture**.

We reproduce the similarity-based importance scoring method introduced in *What Matters in Transformers?* and apply structured layer ablations to three representative models: (i) a Romanian-adapted decoder-only model (RoLLaMA2-7B), (ii) a smaller decoder-only baseline (TinyLlama-1.1B), and (iii) an encoder–decoder translation model (mT5-small). Using importance rankings computed on a small calibration set, we progressively remove attention and MLP layers and evaluate the resulting models on task-specific benchmarks. Our contributions are threefold:

- We evaluate Transformer layer redundancy across models of different scale, language adaptation, and architecture, extending prior analyses beyond large English decoder-only models.
- We provide a systematic comparison of **attention and MLP layer ablations**, showing consistent attention redundancy across settings and stronger sensitivity to MLP removal.
- For encoder–decoder models, we analyze redundancy separately for encoder self-attention, encoder MLP, decoder self-attention, decoder cross-attention, and decoder MLP, highlighting that redundancy is **component-dependent** and that decoder cross-attention is particularly critical.

2 Related Work

Transformer-based LLMs have achieved strong performance across diverse NLP tasks, but their substantial computational and memory requirements create significant deployment challenges in industrial settings where low latency and cost efficiency are critical (Sun et al., 2023; Gromov et al., 2024). These constraints have motivated extensive research into model compression techniques.

2.1 Model Compression Techniques

Quantization lowers precision of model weights, significantly reducing memory usage and inference cost, and has become a common industry practice due to its strong performance–efficiency trade-off (Frantar et al., 2023; Lin et al., 2024). However, quantization preserves the original model architecture and does not address structural redundancy.

Pruning techniques remove parameters or architectural components from the model. Unstructured

pruning eliminates individual weights but results in sparse models that are inefficient on standard hardware (Sanh et al., 2020). In contrast, structured pruning removes entire units such as neurons or layers, producing hardware-friendly architectures but often with greater performance degradation (Zhuang et al., 2020; Kwon et al., 2020).

Knowledge distillation trains a smaller model to approximate a larger one (Hinton et al., 2015), achieving strong compression but requires expensive retraining.

2.2 Redundancy in LLM Architectures

Recent work has moved beyond parameter compression toward analyzing redundancy at the architectural level. Prior studies show that not all Transformer layers contribute equally to performance, with deeper layers often providing diminishing returns (Gromov et al., 2024; Men et al., 2024).

Most notably, (He et al., 2024) demonstrated that attention layers exhibit substantial redundancy, allowing a large fraction of them to be removed with minimal performance loss, and showed that jointly dropping attention and MLP layers enables more aggressive compression.

3 Methodology

3.1 Overview

We reproduce the layer redundancy analysis proposed in *What Matters in Transformers?* and apply it to models of different scales and architectures. We evaluate a Romanian decoder-only model (RoLLaMA-7B), a smaller baseline model (TinyLlama-1.1B), and an encoder–decoder model (mT5-small) to see how layer importance changes depending on the model type.

To estimate which layers matter most, we use the similarity-based calibration method introduced in the original paper. Based on these scores, we apply three structured ablation strategies: *Attention Drop*, *MLP Drop*, and *Joint Layer Drop*. The pruned models are then evaluated on language modeling, reasoning, and translation benchmarks.

3.2 Calibration Process

We follow the calibration procedure described in the original work. A small calibration set is used only for measuring how much each layer transforms its input, without updating any model parameters.

For each layer l, we compute an importance score using the cosine similarity between its input and output hidden states:

$$S_l = 1 - \cos(X_l, Y_l)$$

Layers that change the representation less receive lower scores and are considered more redundant.

3.3 Measuring Layer Importance

In Transformer architectures, each sublayer is wrapped in a residual connection. Formally, for an input X and sublayer function $f(\cdot)$, the output is computed as $Y = X + f(\text{LayerNorm}(X))$. This structure means that even if a sublayer produces minimal transformation, information can still flow through the residual path.

Because of this design, evaluating importance based solely on the sublayer output $f(X)$ would be misleading. Instead, we treat each sublayer together with its associated LayerNorm as a single unit and measure the similarity between the unit's input X and final output Y (after the residual addition). Layers where $X \approx Y$ contribute little to the representation and are considered redundant.

This approach yields separate importance rankings for Attention and MLP layers, which are then used to select candidates for removal under different pruning ratios.

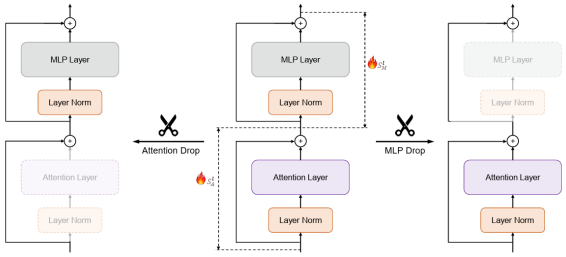


Figure 1: Visualization of structured layer ablation in Transformer blocks. The residual connections ensure that information can still flow through the network after ablation. Figure adapted from He et al. (2024).

3.4 Layer Ablation Strategies

Using the importance rankings obtained during calibration, we apply structured ablation by removing entire sublayer units rather than individual parameters. We consider three strategies:

3.4.1 Attention Drop

Attention Drop removes self-attention layers that exhibit low importance scores. Each attention mod-

ule is treated jointly with its associated LayerNorm due to the residual structure. Redundant attention layers are ablated by zeroing their projection weights, while the residual connection ensures uninterrupted information flow.

3.4.2 MLP Drop

MLP Drop targets feed-forward sublayers, which also operate within residual connections. To obtain a meaningful importance estimate, the MLP layer and its preceding LayerNorm are treated as a single unit. Layers identified as redundant are ablated by zeroing their projection weights.

3.4.3 Joint Drop

Joint Drop combines Attention Drop and MLP Drop by removing redundant layers from both categories in a single step based on their respective importance rankings. This strategy enables more aggressive compression and facilitates the analysis of redundancy across different Transformer components.

4 Experiments & Results

4.1 Decoder-only Model: RoLLaMA2-7B

4.1.1 Experimental Setup

We conduct layer ablation experiments on **RoLLaMA2-7B-Base**, a Romanian-adapted decoder-only Transformer derived from LLaMA2. The model consists of 32 Transformer layers, each containing a self-attention module and an MLP sublayer.

To estimate layer importance, we perform similarity-based calibration using the Romanian Wikipedia corpus (`wikimedia/wikipedia`, `20231101.ro`). We sample 256 text segments of up to 2048 tokens, filtered to ensure sufficient length and content diversity. This calibration set is used exclusively to compute importance scores and does not involve any parameter updates.

Based on the resulting importance rankings, we generate multiple pruned model variants using three structured ablation strategies. For Attention Drop, we remove 4, 8, 12, 16, and 20 most redundant attention layers. For MLP Drop, we remove 4, 8, and 12 most redundant MLP layers. Joint Drop combines both strategies by removing attention and MLP layers simultaneously at increasing drop ratios.

All pruned models are evaluated against the original baseline using zero-shot accuracy on a suite of reasoning and commonsense benchmarks.

4.1.2 Evaluation Benchmarks

All pruned model variants are evaluated using the same benchmark suite as the original, unpruned RoLLaMA2-7B baseline: **ARC-Easy** and **ARC-Challenge** test scientific reasoning, **HellaSwag** evaluates commonsense narrative completion, and **Winogrande** measures pronoun resolution and logical consistency.

All evaluations are conducted using the `lm-eval-harness` framework, which provides standardized implementations and metrics across tasks. Models are evaluated in a zero-shot setting with identical decoding and batching configurations for fair comparison.

4.1.3 Hardware and Runtime

All experiments are conducted in inference-only mode on a single NVIDIA GB10 GPU with 128 GB of VRAM. Models are loaded in half-precision (`float16`) and evaluated without gradient computation.

4.1.4 Attention Layer Drop Results

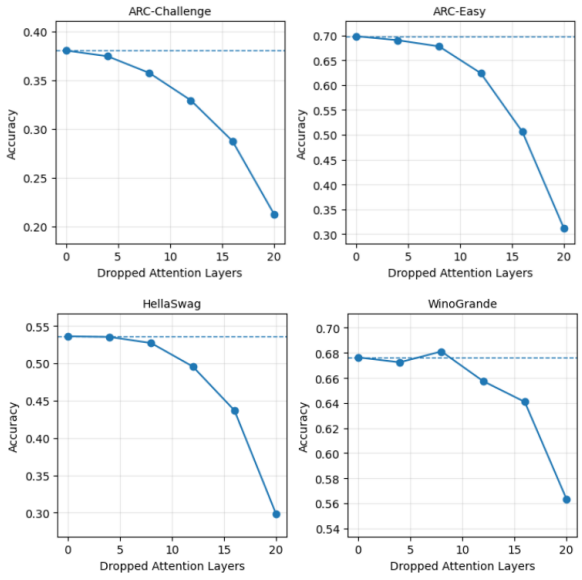


Figure 2: Attention layer drop results on RoLLaMA2-7B across evaluation benchmarks.

Figure 2 illustrates the impact of progressively removing attention layers from RoLLaMA2-7B. Across all benchmarks, performance degrades gradually as attention layers are removed, with only minor drops observed for up to 8 removed layers.

Notably, removing 8 out of 32 attention layers (25%) preserves over 90% of baseline performance

on most tasks. Even at higher pruning levels (12–16 layers), the model maintains reasonable accuracy on HellaSwag and WinoGrande, indicating substantial redundancy in attention modules. In contrast, tasks requiring multi-step reasoning, such as ARC-Challenge, exhibit earlier degradation.

Overall, these results confirm that attention layers can be aggressively pruned with limited impact on downstream performance.

4.1.5 MLP Layer Drop Results

Figure 3 presents the results of MLP layer ablation. In contrast to attention pruning, removing MLP layers leads to a significantly faster performance decline across all benchmarks.

Dropping only 4 MLP layers results in noticeable degradation, particularly on ARC and HellaSwag. At higher pruning levels, performance decreases sharply, suggesting that MLP layers play a more critical role in preserving model capacity.

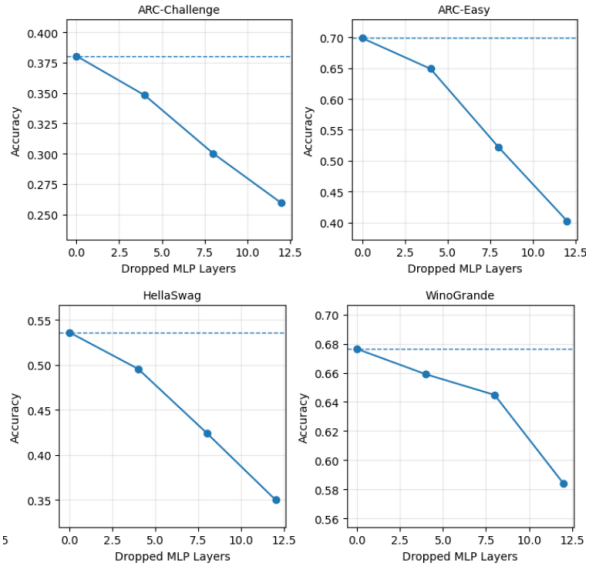


Figure 3: MLP layer drop results on RoLLaMA2-7B across evaluation benchmarks.

4.1.6 Joint Layer Drop Results

Joint layer drop combines attention and MLP pruning to achieve more aggressive compression. As shown in Figure 4, joint pruning enables larger reductions in model size but introduces earlier performance collapse.

For moderate pruning levels (up to 16 dropped sub-blocks), the model retains competitive performance on several benchmarks. However, beyond this point, accuracy deteriorates rapidly, especially on ARC-Easy and HellaSwag, indicating that redundancy limits have been exceeded.

Joint drop thus provides a flexible trade-off between compression and performance but requires careful calibration to avoid catastrophic degradation.

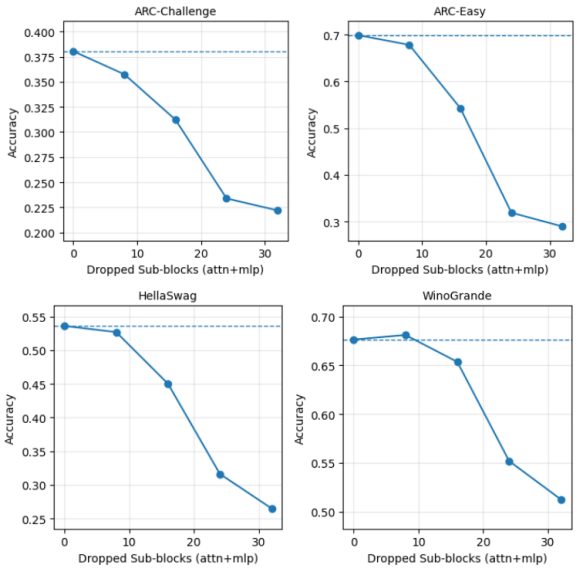


Figure 4: Joint attention and MLP layer drop results on RoLLaMA2-7B across evaluation benchmarks.

4.1.7 Summary of Findings

Method	ARC-C	ARC-E	HellaSwag	WinoGrande
Baseline	0.381	0.699	0.536	0.676
Attn-4	0.375	0.691	0.535	0.673
Attn-8	0.358	0.678	0.527	0.681
Attn-12	0.329	0.624	0.496	0.658
Attn-16	0.288	0.506	0.437	0.641
Attn-20	0.213	0.311	0.298	0.564
MLP-4	0.348	0.649	0.495	0.659
MLP-8	0.300	0.522	0.424	0.645
MLP-12	0.259	0.402	0.350	0.584
Joint-8	0.358	0.678	0.527	0.681
Joint-16	0.312	0.543	0.450	0.654
Joint-24	0.234	0.319	0.316	0.552
Joint-32	0.222	0.290	0.265	0.512

Table 1: Summary of RoLLaMA2-7B performance under different layer ablation strategies. Scores are reported as accuracy for each benchmark.

Across all benchmarks, Attention Drop preserves performance better than MLP Drop for the same number of removed layers, indicating higher redundancy in attention layers. In contrast, MLP ablations cause faster degradation, reflecting their greater impact on model capacity. Joint Drop allows more aggressive compression, but leads to a sharp accuracy collapse beyond moderate pruning levels.

4.2 Decoder-only Model: TinyLlama-1.1B

4.2.1 Experimental Setup

We analyze layer redundancy on **TinyLlama-1.1B** (Team, 2023), a compact 22-layer decoder-only Transformer, to verify whether the attention-layer redundancy also appears at smaller model scale.

Using the same similarity-based calibration pipeline, we generate pruned variants under three ablation strategies: *Attention Drop*, *MLP Drop*, and *Joint Drop*. We evaluate pruning ratios $r \in \{10\%, 25\%, 50\%\}$. For Attention and MLP Drop, these correspond to removing approximately $\{2, 6, 11\}$ layers from the 22-layer stack. For Joint Drop, we rank attention and MLP units together, resulting in 44 candidate modules, and remove $\{4, 11, 22\}$ modules in total.

Calibration corpora and analysis. To study how the estimated layer importance depends on the calibration data, we compute similarity-based scores on four corpora: **wiki_ro**, **wiki_en**, **mC4_ro**, and **mC4_en**. Each corpus provides 256 samples truncated to 2048 tokens. The Wikipedia splits (Romanian and English) contain clean encyclopedic text, while the mC4 splits come from a broader and noisier web domain. We compute layer-importance scores on these corpora to estimate how redundant each Transformer layer is. Lower scores indicate layers that modify the representation less and are therefore more suitable for removal.

Across all calibration sets, MLP layers consistently have higher median importance scores than attention layers. For example, on **wiki_ro**, the median score is 0.044 for attention layers and 0.124 for MLP layers, which indicates that attention modules are generally more redundant.

We also analyze how stable these rankings are across corpora using Spearman correlation. The attention-layer rankings remain highly consistent (between 0.80 and 0.97), even across languages and domains. This suggests that attention redundancy is largely an intrinsic property of TinyLlama.

In contrast, MLP-layer rankings are slightly less stable, meaning that their relative importance varies more depending on the calibration data.

Ablation variants. For each dropping strategy (Attention, MLP, and Joint), we build pruned model variants using the layer-importance ranking obtained from the wiki_en calibration set. The remaining calibration corpora are used only to assess the cross-lingual and cross-domain stability of these rankings. This setup allows us to verify whether the redundant-layer ordering reflects an intrinsic property of the model or depends on the calibration distribution.

4.2.2 Evaluation Benchmarks

We evaluate both the baseline and the pruned TinyLlama models in a zero-shot setting using the LM Harness framework. We use four standard reasoning benchmarks: **BoolQ**, **HellaSwag**, **PIQA**, and **WinoGrande**. For each model, we report the accuracy on every task, as well as an overall score computed as the simple average across all four benchmarks.

Perplexity evaluation. In addition to benchmark accuracy, we measure language modeling quality by computing perplexity on a held-out Romanian mC4 split. Perplexity is calculated as the standard cross-entropy loss over 2048-token sequences, providing a direct estimate of how pruning affects the model's generative ability.

4.2.3 Hardware and Runtime

All the experiments were run on Google Colab using a single NVIDIA A100 GPU (40GB VRAM). We evaluate models in float16 precision with the same batch size across all configurations to ensure fair runtime comparison.

4.2.4 Attention Layer Drop Results

Figure 5 illustrates how TinyLlama's performance changes as we remove attention layers. We can see that dropping a small number of attention layers (up to about 25%) has very little effect on the model's accuracy. Tasks like PIQA and WinoGrande remain almost unchanged even when we remove half of the attention layers, suggesting that many of these layers are not essential. On the other hand, BoolQ and HellaSwag start to lose performance more clearly at higher drop ratios.

This behavior is consistent with our calibration analysis, which shows that the most redundant attention layers are located in the deeper part of the network. A similar pattern appears in the language modeling metric: attention pruning results in only a moderate increase in perplexity (from 14.6 to around 23 at 25% drop), confirming that removing deeper attention layers has limited impact on overall model quality.

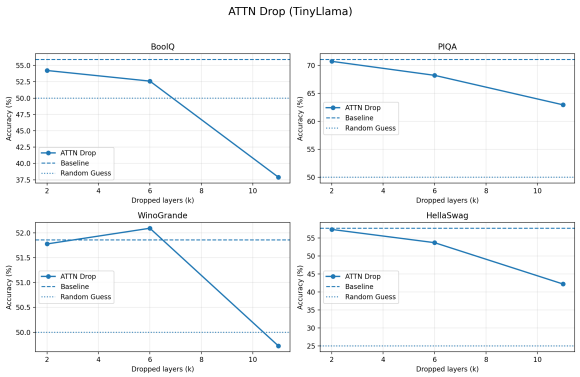


Figure 5: Impact of Attention Layer Dropping on TinyLlama-1.1B.

4.2.5 MLP Layer Drop Results

Figure 6 shows how TinyLlama's accuracy changes when we remove MLP layers. Unlike Attention Drop, removing MLP layers leads to a much stronger performance decrease across all benchmarks. For example, PIQA and HellaSwag show a clear and almost linear drop as more MLP layers are removed. BoolQ presents some fluctuations, but its overall trend also declines once we drop a larger number of layers.

We also observe a much stronger increase in perplexity when MLP layers are removed, compared to attention pruning. This suggests that MLP layers are more important for preserving the model's language modeling ability.

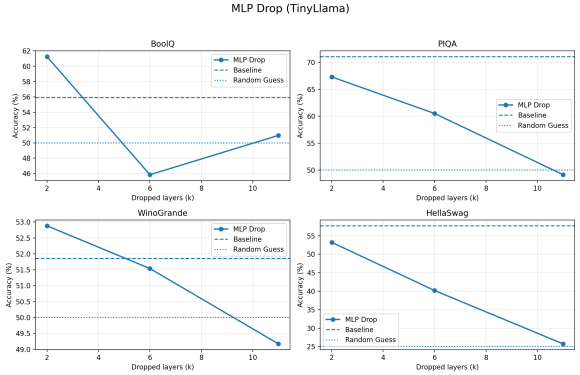


Figure 6: TinyLlama: performance after dropping k MLP layers (selected by the calibration ranking).

4.2.6 Joint Drop Results

Figure 7 shows how TinyLlama's performance changes when we remove both attention and MLP modules at the same time. Unlike the previous experiments where we dropped only one type of layer, here we observe that the model's accuracy decreases more quickly as we increase the drop ratio.

For small pruning values, the model still performs close to the baseline, which suggests that some modules are redundant. However, when more modules are removed, the drop in performance becomes more noticeable across all tasks. PIQA and WinoGrande remain more stable, while BoolQ and HellaSwag are affected more strongly, meaning they depend more on deeper layers of the model.

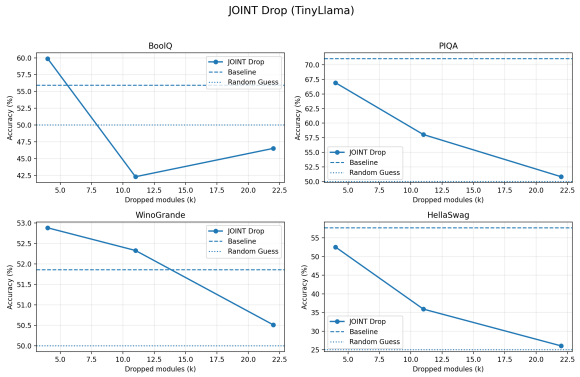


Figure 7: Effect of Joint Layer Dropping on TinyLlama across four benchmarks (BoolQ, PIQA, WinoGrande, and HellaSwag).

4.2.7 Summary of Findings

Table 2 summarizes the impact of structured layer dropping on TinyLlama-1.1B. Across all four evaluation tasks, Attention Drop consistently preserves performance better than MLP Drop for the same pruning ratios.

Method	BoolQ	PIQA	WinoGrande	HellaSwag
Baseline	55.90	71.06	51.85	57.69
Attn-10	54.22	70.73	51.78	57.36
Attn-25	52.60	68.23	52.09	53.68
Attn-50	37.89	62.95	49.72	42.19
MLP-10	61.25	67.30	52.88	53.22
MLP-25	45.84	60.50	51.54	40.18
MLP-50	50.98	49.13	49.17	25.74
Joint-10	59.88	66.92	52.88	52.53
Joint-25	42.29	58.05	52.33	35.92
Joint-50	46.51	50.82	50.51	26.06

Table 2: TinyLlama-1.1B accuracy after structured layer dropping.

Removing 10% of attention layers leads to almost no degradation, and even at 25% pruning the model remains close to the baseline on all four tasks. In contrast, MLP Drop shows a different behavior: the model is relatively stable at low

drop ratios, but performance decreases much more abruptly once more MLP layers are removed. This suggests that MLP layers are less redundant and become critical when pruned more aggressively.

Joint Drop combines both types of pruning, but in our experiments it does not bring the same benefit. While performance is still acceptable at 10% pruning, increasing the drop ratio leads to a clear degradation across all benchmarks, especially on BoolQ and HellaSwag. Overall, TinyLlama tolerates attention pruning better than MLP or joint pruning, highlighting that attention layers remain the most redundant component in this model.

4.3 Seq2Seq Translation: mT5-small

4.3.1 Experimental Setup

We evaluate layer redundancy on **mT5-small**, a multilingual encoder–decoder Transformer with 8 encoder and 8 decoder layers, fine-tuned for Romanian→English translation.

Dataset. We use WMT16 Ro–En with 20,000 training pairs and 1,000 validation pairs, plus a separate 512-sample calibration split drawn from the shuffled training subset (seed=42) for importance scoring. Inputs are prefixed with `translate Romanian to English:` and truncated to 128 tokens (targets are capped at 128 tokens for baseline training).

Baseline. The model is fine-tuned for 3 epochs (AdamW, lr $= 5 \times 10^{-5}$, per-device batch size 8, gradient accumulation 2). Decoding uses beam search with 4 beams. We select the checkpoint with the highest validation SacreBLEU, achieving 16.90 SacreBLEU on the validation set.

Ablation protocol. We evaluate two settings: (i) **D0**, direct evaluation after dropping $k \in \{1, 2, 3\}$ layers without retraining; and (ii) **D1**, recovery fine-tuning for 1 or 2 epochs (denoted `D1_1e` and `D1_2e`) on the same 20k training subset.

Evaluation metrics. We report translation quality using **SacreBLEU** and **chrF++** (character n-gram F-score). SacreBLEU is used for checkpoint selection on the validation set, while both SacreBLEU and chrF++ are reported for all ablation configurations.

4.3.2 Compute

All experiments are conducted on Google Colab using a single NVIDIA T4 GPU (15 GB VRAM). Baseline fine-tuning (3 epochs) takes approximately 1.25 hours. D0 evaluations take about ≈3 minutes per configuration. D1 recovery fine-tuning takes ≈15–18 minutes per epoch.

4.3.3 Encoder Ablations

Importance ranking. We compute importance scores on the 512-sample calibration set and rank encoder blocks within each component family. The three least important encoder self-attention layers are **3, 6, 7**, while for encoder MLP the three least important layers are **5, 3, 6** (layer indices are 0-based).

D0 results. Table 3 reports performance after dropping k layers without retraining.

Table 3: Encoder D0: dropping k layers without recovery.

Component	k	BLEU	chrF++
Baseline	0	16.90	41.72
Self-Attn	1	15.97	40.66
Self-Attn	2	13.49	37.07
Self-Attn	3	8.09	29.39
MLP	1	16.27	41.17
MLP	2	12.60	38.19
MLP	3	10.32	35.85

At $k = 1$, dropping MLP layers degrades less than dropping self-attention layers (16.27 vs. 15.97 BLEU). At $k = 3$, both components degrade substantially, with self-attention dropping more severely (8.09 vs. 10.32 BLEU).

D1 results. Table 4 shows recovery after 1–2 epochs of fine-tuning.

Table 4: Encoder D1: recovery fine-tuning after dropping k layers.

Component	k	D1_1e		D1_2e	
		BLEU	chrF++	BLEU	chrF++
Baseline	0	17.87	43.05	18.72	43.81
Self-Attn	1	17.37	42.54	17.97	43.43
Self-Attn	2	16.74	42.47	16.98	43.27
Self-Attn	3	15.24	41.90	15.46	42.53
MLP	1	17.12	41.94	17.52	42.52
MLP	2	16.15	40.74	16.77	41.68
MLP	3	15.69	39.91	16.43	41.18

Recovery effect. Figure 8 summarizes the recovery trend by comparing D0 (no recovery) to D1_2e

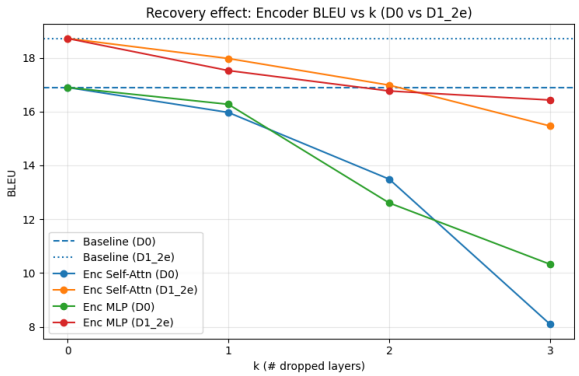


Figure 8: Recovery effect on the encoder (BLEU vs. k): direct evaluation after dropping layers (D0) compared to recovery fine-tuning (D1_2e). Dashed lines indicate the corresponding baselines; solid lines show performance after dropping $k \in \{1, 2, 3\}$ least-important layers within each component family.

(2 epochs of recovery fine-tuning) for both encoder self-attention and encoder MLP. Recovery substantially closes the gap: after 2 epochs, dropping 3 self-attention layers reaches 15.46 BLEU (vs. 8.09 at D0), while dropping 3 MLP layers reaches 16.43 BLEU (vs. 10.32 at D0).

Key finding. In the encoder, both self-attention and MLP layers are sensitive to aggressive dropping, but recovery fine-tuning mitigates most of the degradation. Under D0, MLP layers are slightly more resilient at small k, while under D1 both component families recover to comparable performance.

4.3.4 Decoder Ablations

Importance ranking. We compute importance scores on the 512-sample calibration set and rank decoder blocks within each component family. The least important layers (0-based indices) differ across decoder components; we report the top-ranked redundant layers for decoder self-attention, cross-attention, and MLP in Appendix **??**.

D0 results. Table 5 reports performance after dropping k layers without retraining for each decoder component. Dropping cross-attention layers degrades the most, indicating that cross-attention is the most critical decoder component for translation.

At $k = 1$, self-attention is relatively robust (16.73 BLEU), while cross-attention shows a larger drop (15.14 BLEU). For larger k, cross-attention degrades sharply (down to 7.89 BLEU at $k = 3$), whereas self-attention declines moderately and MLP exhibits intermediate sensitivity.

Table 5: Decoder D0: dropping k layers without recovery.

Component	k	BLEU	chrF++
Baseline	0	16.90	41.72
Self-Attn	1	16.73	41.83
Self-Attn	2	16.13	41.20
Self-Attn	3	15.89	40.98
Cross-Attn	1	15.14	39.15
Cross-Attn	2	9.34	30.49
Cross-Attn	3	7.89	26.42
MLP	1	15.27	40.56
MLP	2	10.81	35.34
MLP	3	10.68	35.06

D1 results. Table 6 reports recovery fine-tuning results for the $k = 1$ setting (best-k configuration available for D1 in our runs). Recovery improves all components, but cross-attention remains the most sensitive.

Table 6: Decoder D1: recovery fine-tuning after dropping $k = 1$ layer (best-k).

	D1_1e		D1_2e	
Component	BLEU	chrF++	BLEU	chrF++
Baseline	17.87	43.05	18.72	43.81
Self-Attn (k=1)	17.40	42.27	18.32	43.34
Cross-Attn (k=1)	16.07	41.81	16.72	42.51
MLP (k=1)	17.44	42.23	18.06	43.22

D0 comparison plot. Figure 9 visualizes BLEU vs. k under D0 for the three decoder components, highlighting the steep degradation when dropping cross-attention layers.

Key finding. Decoder cross-attention is markedly less redundant than decoder self-attention and decoder MLP: dropping even a small number of cross-attention layers causes substantial quality loss, and recovery fine-tuning does not fully close the gap. In contrast, decoder self-attention tolerates moderate dropping with relatively small degradation.

4.3.5 Encoder–Decoder Comparison

To directly compare sensitivity across encoder and decoder components, we summarize performance under the most aggressive ablation setting ($k = 3$) in the **D0** regime (no recovery). Figure 10 shows the resulting BLEU scores for encoder self-

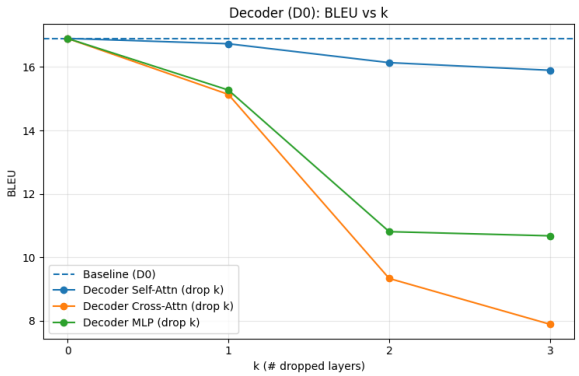


Figure 9: Decoder D0 (no recovery): BLEU vs. k when dropping decoder self-attention, cross-attention, or MLP layers. Cross-attention is the most critical component, with performance collapsing as k increases.

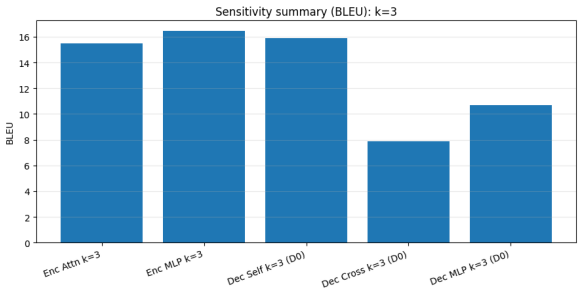


Figure 10: Sensitivity summary at $k = 3$ (D0, no recovery): BLEU after dropping the three least important layers within each component family. Encoder components remain relatively robust, while dropping decoder cross-attention leads to a sharp performance collapse.

attention, encoder MLP, and the three decoder component families.

Overall, the encoder is comparatively more robust to dropping: at $k = 3$, encoder self-attention and encoder MLP retain ≈15–16 BLEU. In contrast, the decoder exhibits much higher sensitivity when cross-attention layers are removed, with performance collapsing to **7.89 BLEU**. Decoder self-attention is substantially more tolerant (15.89 BLEU), while decoder MLP shows intermediate degradation (10.68 BLEU). These results highlight that, for encoder–decoder translation models, **decoder cross-attention is the least redundant component** and dominates performance under aggressive layer dropping.

4.3.6 Summary of Findings

- **Recovery works:** D1 fine-tuning substantially mitigates the large D0 degradation from dropping encoder layers.
- **Encoder sensitivity is similar across components:** self-attention and MLP show comparable robustness, with only small differences at low k.
- **Decoder cross-attention is critical:** dropping cross-attention causes the sharpest quality collapse, far more than decoder self-attention or MLP.
- **Redundancy is asymmetric:** encoder components tolerate aggressive dropping better than decoder cross-attention, which dominates performance loss.

5 Conclusion & Discussion

Our hypothesis that attention layers are more redundant than MLP layers in Transformer architectures is largely confirmed by the experimental results. Across both decoder-only models, RoLLaMA2-7B and TinyLlama-1.1B, we observe that pruning attention layers leads to only gradual performance degradation, whereas removing MLP layers produces a noticeably faster decline in benchmark accuracy and language modeling quality. These results support prior findings by He et al. (2024), who reported similar redundancy patterns in large English LLMs, and demonstrate that the same phenomenon holds for Romanian-adapted and smaller-scale models.

At the same time, our encoder–decoder experiments reveal that redundancy is not uniform across Transformer components. While encoder layers can recover most of their performance after brief fine-tuning, decoder cross-attention layers are far less redundant and represent a critical bottleneck for translation quality. This highlights an important architectural asymmetry and suggests that pruning strategies should be applied selectively depending on the model type and downstream task.

6 Limitations

Our experiments cover only a few models and tasks, and rely on similarity-based importance scoring over small calibration sets. We do not report full runtime or memory improvements. Future work should evaluate redundancy on larger datasets, across more architectures, and with detailed efficiency metrics.

References

Elias Frantar, Saleh Ashkboos, Torsten Hoefler, and Dan Alistarh. 2023. Gptq: Accurate post-training

quantization for generative pre-trained transformers. *arXiv preprint arXiv:2210.17323*.

Andrey Gromov, Kushal Tirumala, Hassan Shapourian, Paolo Glorioso, and Daniel A. Roberts. 2024. The unreasonable ineffectiveness of the deeper layers. *arXiv preprint arXiv:2403.17887*.

Shwai He, Guoheng Sun, Zhenyu Shen, and Ang Li. 2024. What matters in transformers? not all attention is needed. *arXiv preprint arXiv:2406.15786*.

Geoffrey Hinton, Oriol Vinyals, and Jeff Dean. 2015. Distilling the knowledge in a neural network. *arXiv preprint arXiv:1503.02531*.

Se Jung Kwon, Dongsoo Lee, Byeongwook Kim, Parichay Kapoor, Baeseong Park, and Gu-Yeon Wei. 2020. Structured compression by weight encryption for unstructured pruning and quantization. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 1906–1915.

Ji Lin, Jiaming Tang, Haotian Tang, Shang Yang, Wei-Ming Chen, Wei-Chen Wang, Guangxuan Xiao, Xingyu Dang, Chuang Gan, and Song Han. 2024. Awq: Activation-aware weight quantization for llm compression and acceleration. In *MLSys*.

Xin Men, Mingyu Xu, Qingyu Zhang, Bingning Wang, Hongyu Lin, Yaojie Lu, Xianpei Han, and Weipeng Chen. 2024. Shortgpt: Layers in large language models are more redundant than you expect. *arXiv preprint arXiv:2403.03853*.

Victor Sanh, Thomas Wolf, and Alexander M. Rush. 2020. Movement pruning: Adaptive sparsity by fine-tuning. *arXiv preprint arXiv:2005.07683*.

Mingjie Sun, Zhuang Liu, Anna Bair, and J. Zico Kolter. 2023. A simple and effective pruning approach for large language models. *arXiv preprint arXiv:2306.11695*.

TinyLlama Team. 2023. Tinyllama: An open and efficient language model. https://huggingface.co/TinyLlama/TinyLlama-1.1B-Chat-v1.0.

Tao Zhuang, Zhixuan Zhang, Yuheng Huang, Xiaoyi Zeng, Kai Shuang, and Xiang Li. 2020. Neuron-level structured pruning using polarization regularizer. In *Advances in Neural Information Processing Systems*, volume 33, pages 9865–9877.